

# Views and Access Control Exercise - GameVerse Database

---

## Practical Exercise: Role-Based View Access

**Duration:** 60 minutes

**Database:** GameVerse (Game Store Database)

---

## Learning Objectives

By the end of this exercise, you will:

- Create database views that join multiple tables
  - Understand how views provide a security layer
  - Create roles and users with specific permissions
  - Grant view-level permissions without underlying table access
  - Test and verify role-based access control
  - Answer the key question: **Do users need access to underlying tables when querying views?**
- 

## Important Concept: Views and Security

Key Question: Do you need to grant users access to underlying tables when they query a view?

**Answer: NO!**

This is one of the most powerful features of views for access control. Users can query a view **without having direct access to the underlying tables**. The view executes with the permissions of the view owner (typically the database owner or admin who created the view).

Why is this useful?

**Column-level security:** Expose only specific columns from a table (hide sensitive data)

**Row-level filtering:** Show only certain rows (e.g., active users only)

**Data masking:** Transform sensitive data before showing it

**Simplified access:** Join multiple tables and present unified data

**Business logic:** Implement complex rules without exposing raw tables

**Easier management:** Change one view instead of managing many table permissions

---

## Prerequisites

Before starting, ensure:

- PostgreSQL is installed and running
- The GameVerse database is created and populated with sample data

- You have admin/superuser access to create roles and views
  - You can open multiple terminal/psql sessions for testing
- 

## Part 1: Create Views with Joins (20 minutes)

### Exercise 1.1: Create a Sales Analytics View

Create a view called `sales_report_view` that provides sales analytics data by joining multiple tables.

**Requirements:** The view should show:

- Game title
- Publisher name
- Genre name
- Total copies sold (count of purchases in `user_library`)
- Total revenue (sum of `purchase_price` from `user_library`)
- Average price per sale

The data should be grouped by game.

**Your SQL:**

```
CREATE VIEW sales_report_view AS
-- Write your query here
```

---

### Exercise 1.2: Test Your Sales View

Run a query to test the view:

```
SELECT * FROM sales_report_view LIMIT 10;
```

---

**Questions:**

1. How many rows does the view return?
  2. Which game has the highest total revenue?
  3. Are there any games with zero sales?
- 

### Exercise 1.3: Create a User Activity View

Create a view called `user_activity_view` that shows user engagement metrics.

**Requirements:** The view should show:

- User ID
- Username

- Email (masked: show only first 3 characters, e.g., "abc\*\*\*@\*\*\*")
- Registration date
- Total games owned (count from user\_library)
- Total hours played (sum from user\_library)
- Total reviews written (count from reviews)
- Total achievements unlocked (count from user\_achievements)
- Account status
- Premium status (is\_premium)

**Your SQL:**

```
CREATE VIEW user_activity_view AS
-- Write your query here
```

---

**Exercise 1.4: Test Your User Activity View**

Run a query to test the view:

```
SELECT * FROM user_activity_view LIMIT 10;
```

**Questions:**

1. How is the email displayed? Is the full email visible?
2. Which user has the most hours played?
3. Can you see which user has the most achievements?

---

**Part 2: Create Roles and Users (10 minutes)****Exercise 2.1: Create Two Roles**

Create two separate roles that will have different access permissions:

1. **sales\_analyst** - Will have access to sales data only
2. **community\_manager** - Will have access to user activity data only

Both roles should have LOGIN capability.

**Your SQL:**

```
-- Create sales_analyst role
CREATE ROLE sales_analyst -- Complete this

-- Create community_manager role
CREATE ROLE community_manager -- Complete this
```

---

## Exercise 2.2: Create Two Test Users

Create two database users that will be assigned to the roles:

1. **alice\_sales** - Will be assigned the sales\_analyst role
2. **bob\_community** - Will be assigned the community\_manager role

**Your SQL:**

```
-- Create alice_sales user
CREATE USER alice_sales -- Complete this

-- Create bob_community user
CREATE USER bob_community -- Complete this
```

---

## Exercise 2.3: Verify Roles and Users Exist

Write a query to list all roles and check if your new roles/users were created:

```
SELECT rolname, rolcanlogin
FROM pg_roles
WHERE rolname IN ('sales_analyst', 'community_manager', 'alice_sales',
'bob_community');
```

**Expected Output:** You should see 4 rows with all roles having `rolcanlogin = true`.

---

## Part 3: Grant Permissions (15 minutes)

### Exercise 3.1: Grant Basic Database Access

Both roles need basic access to connect to the database and use the schema.

Grant the following permissions to BOTH roles:

- CONNECT privilege on the `gameverse` database
- USAGE privilege on the `public` schema

**Your SQL:**

```
-- Grant CONNECT on database
GRANT CONNECT ON DATABASE gameverse TO -- Complete this

-- Grant USAGE on schema
GRANT USAGE ON SCHEMA public TO -- Complete this
```

---

## Exercise 3.2: Grant View-Specific Permissions

Now grant each role access to their specific view only:

- `sales_analyst` should have SELECT permission ONLY on `sales_report_view`
- `community_manager` should have SELECT permission ONLY on `user_activity_view`

⚠ **Important:** Do NOT grant any permissions on the underlying tables (games, users, publishers, etc.)

**Your SQL:**

```
-- Grant SELECT on sales_report_view to sales_analyst
GRANT SELECT ON -- Complete this

-- Grant SELECT on user_activity_view to community_manager
GRANT SELECT ON -- Complete this
```

---

## Exercise 3.3: Assign Roles to Users

Assign each role to its corresponding user:

- Assign `sales_analyst` role to `alice_sales`
- Assign `community_manager` role to `bob_community`

**Your SQL:**

```
-- Assign sales_analyst to alice_sales
GRANT -- Complete this

-- Assign community_manager to bob_community
GRANT -- Complete this
```

---

## Part 4: Test Access Control (15 minutes)

Now comes the critical part: verifying that each user can ONLY access their designated view!

### Exercise 4.1: Test as `alice_sales` (Sales Analyst)

Open a new terminal window or psql session and connect as `alice_sales`:

```
psql -U alice_sales -d gameverse -h localhost
# When prompted, enter password: alice123
```

Now run these queries one by one and **document the results**:

```
-- Test 1: Try to query the sales view (SHOULD WORK)
SELECT * FROM sales_report_view LIMIT 5;

-- Test 2: Try to query the user activity view (SHOULD FAIL)
SELECT * FROM user_activity_view LIMIT 5;

-- Test 3: Try to query the games table directly (SHOULD FAIL)
SELECT * FROM games LIMIT 5;

-- Test 4: Try to query the users table directly (SHOULD FAIL)
SELECT * FROM users LIMIT 5;

-- Test 5: Try to query the publishers table directly (SHOULD FAIL)
SELECT * FROM publishers LIMIT 5;
```

Document your results:

| Query                      | Expected Result | Actual Result | Error Message (if any) |
|----------------------------|-----------------|---------------|------------------------|
| Test 1: sales_report_view  | SUCCESS         |               |                        |
| Test 2: user_activity_view | FAIL            |               |                        |
| Test 3: games table        | FAIL            |               |                        |
| Test 4: users table        | FAIL            |               |                        |
| Test 5: publishers table   | FAIL            |               |                        |

Exercise 4.2: Test as bob\_community (Community Manager)

Open another terminal window or psql session and connect as bob\_community:

```
psql -U bob_community -d gameverse -h localhost
# When prompted, enter password: bob123
```

Now run these queries one by one and document the results:

```
-- Test 1: Try to query the user activity view (SHOULD WORK)
SELECT * FROM user_activity_view LIMIT 5;

-- Test 2: Try to query the sales view (SHOULD FAIL)
SELECT * FROM sales_report_view LIMIT 5;

-- Test 3: Try to query the users table directly (SHOULD FAIL)
SELECT * FROM users LIMIT 5;

-- Test 4: Try to query the reviews table directly (SHOULD FAIL)
```

```
SELECT * FROM reviews LIMIT 5;

-- Test 5: Try to see full email addresses (Check if emails are masked)
SELECT username, masked_email FROM user_activity_view LIMIT 5;
```

Document your results:

| Query                      | Expected Result | Actual Result | Error Message (if any) |
|----------------------------|-----------------|---------------|------------------------|
| Test 1: user_activity_view | SUCCESS         |               |                        |
| Test 2: sales_report_view  | FAIL            |               |                        |
| Test 3: users table        | FAIL            |               |                        |
| Test 4: reviews table      | FAIL            |               |                        |
| Test 5: email masking      | Emails masked   |               |                        |

Exercise 4.3: Verify Permissions (Admin)

Switch back to your admin/superuser connection and run these verification queries:

```
-- Query 1: List all views in the database
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'VIEW'
ORDER BY table_name;

-- Query 2: Check what tables/views sales_analyst can access
SELECT table_name, privilege_type
FROM information_schema.table_privileges
WHERE grantee = 'sales_analyst'
ORDER BY table_name, privilege_type;

-- Query 3: Check what tables/views community_manager can access
SELECT table_name, privilege_type
FROM information_schema.table_privileges
WHERE grantee = 'community_manager'
ORDER BY table_name, privilege_type;

-- Query 4: Check role membership for alice_sales
SELECT
    r.rolname AS user_name,
    m.rolname AS member_of_role
FROM pg_roles r
JOIN pg_auth_members am ON r.oid = am.member
JOIN pg_roles m ON am.roleid = m.oid
WHERE r.rolname = 'alice_sales';

-- Query 5: Check role membership for bob_community
SELECT
```

```
    r.rolname AS user_name,  
    m.rolname AS member_of_role  
FROM pg_roles r  
JOIN pg_auth_members am ON r.oid = am.member  
JOIN pg_roles m ON am.roleid = m.oid  
WHERE r.rolname = 'bob_community';
```

### Expected Results:

- Query 1: Should list your two views (and any others created)
- Query 2: Should show only **sales\_report\_view** with SELECT privilege
- Query 3: Should show only **user\_activity\_view** with SELECT privilege
- Query 4: Should show alice\_sales is a member of sales\_analyst
- Query 5: Should show bob\_community is a member of community\_manager

---

## Part 5: Understanding the Concepts (Questions)

### Question 5.1: View Permissions

Why can Alice query **sales\_report\_view** without having access to the **games**, **publishers**, **genres**, or **user\_library** tables?

---

### Question 5.2: View Ownership

What would happen if you revoked SELECT permission on the **games** table from the database owner who created the view?

---

### Question 5.3: Adding More Detail

If you wanted Bob to see more details in the user activity data (like full email addresses instead of masked ones), what's the BEST approach?

Options:

- a) Grant Bob direct SELECT access to the **users** table
- b) Modify **user\_activity\_view** to show full emails for everyone
- c) Create a NEW view with more details and grant Bob access to that
- d) Give Bob superuser privileges

---

### Question 5.4: Benefits of Views for Access Control

List at least 5 benefits of using views for access control instead of granting direct table access.

---

## Part 6: Challenge Exercise (Optional)

### Challenge 6.1: Create a Marketing Analytics View



Design and implement a complete access control system for a marketing team.

### Requirements:

1. Create a view called `marketing_analytics_view` that shows:
  - Game title and price
  - Publisher name
  - Developer name
  - Genre
  - Platform names (comma-separated list)
  - Average review rating
  - Total number of reviews
  - Total sales (copies sold)
  - Release date
  - **BUT NOT**: individual user data, specific reviews, or user emails
2. Create a role called `marketing_team`
3. Create a test user called `maria_marketing`
4. Grant appropriate permissions:
  - Connect to database
  - Use schema
  - SELECT on the marketing view ONLY
5. Assign the role to the user
6. Test the access:
  - Verify `maria_marketing` can query the view
  - Verify `maria_marketing` CANNOT query underlying tables
  - Verify `maria_marketing` CANNOT see other views

### Your Solution:

```
-- Step 1: Create the view
CREATE VIEW marketing_analytics_view AS
-- Your query here (HINT: You'll need to join games, publishers,
-- genres, game_platforms, platforms, reviews, and user_library)

-- Step 2: Create role and user

-- Step 3: Grant permissions

-- Step 4: Assign role to user
```

```
-- Step 5: Test (in a new psql session as maria_marketing)
```

---

## Summary & Key Takeaways

### What You Learned

#### 1. Views as Security Layers

- Views can restrict access to specific columns and rows
- Views execute with owner's permissions
- Users don't need access to underlying tables

#### 2. Role-Based Access Control

- Create roles with specific permissions
- Assign roles to users for easier management
- Each role has access to only their designated views

#### 3. Data Protection

- Mask sensitive data (emails, personal info)
- Hide financial details from unauthorized users
- Expose aggregated data without revealing individuals

#### 4. Permission Management

- GRANT/REVOKE on views, not tables
- Verify permissions with system catalogs
- Test access from multiple user perspectives

### Best Practices Reinforced

- **Principle of Least Privilege:** Give minimum necessary access
- **Separation of Duties:** Different roles for different functions
- **Defense in Depth:** Multiple security layers (roles + views + permissions)
- **Auditability:** Track who can access what through structured views
- **Maintainability:** Change views instead of many user permissions

---

## Submission Checklist

Before submitting, verify:

- ☐ Both views created successfully (sales\_report\_view, user\_activity\_view)
- ☐ Both views use joins (at least 2 tables each)
- ☐ Both roles created (sales\_analyst, community\_manager)
- ☐ Both users created (alice\_sales, bob\_community)

- ☐ Database connection and schema permissions granted
  - ☐ View-specific permissions granted correctly
  - ☐ Roles assigned to users
  - ☐ Access tests completed and documented
  - ☐ All questions answered
  - ☐ Understanding of view permissions vs table permissions
- 

## Additional Resources

- PostgreSQL Documentation: [Views](#)
  - PostgreSQL Documentation: [Privileges](#)
  - PostgreSQL Documentation: [Roles](#)
- 

**Congratulations!** You now understand how to use views as powerful access control mechanisms in PostgreSQL!